

1. Понятие сохраненной процедуры.

Сохраненная процедура – это именованная последовательность SQL-выражений и/или управляющих операторов.

При создании сохраненной процедуры формируется и сохраняется план ее выполнения. В дальнейшем при обращениях к этой процедуре используется готовый план, без повторной компиляции SQL-выражений (кроме случаев использования директивы **with recompile**, которая будет рассмотрена ниже). Таким образом, запуск готовой процедуры производится гораздо быстрее запуска эквивалентного SQL-пакета (batch).

Кроме получения преимущества в скорости запуска, использование сохраненных процедур позволяет гибко настраивать допуски пользователей, давая им разрешение на запуск строго определенных процедур.

И, естественно, подобно функциям в «традиционных» языках программирования, сохраненные процедуры позволяют структурировать приложение, вести разработку по модульному принципу, повышая тем самым надежность программ.

Сохраненная процедура может:

- принимать параметры
- вызывать другие процедуры
- возвращать вызывающей процедуре или клиентскому приложению статусное значение
- возвращать вызывающей процедуре или клиентскому приложению выходные параметры
- возвращать вызывающему клиентскому приложению один или более ResultSet'ов
- быть запущена с удаленных серверов

2. Создание сохраненной процедуры.

Создание сохраненной процедуры производится при помощи инструкции **create procedure**:

```
create proc[edure] [owner.]procedure_name[;number]
    [[(@parameter_name datatype [= default_value][out[put]]
    [, @parameter_name datatype [= default_value][out[put]]]...[)])]
[with recompile]
as { SQL_statement | external name dll_name }
```

Здесь

- **procedure_name** - имя процедуры
- **number** - номер процедуры в группе при использовании групповых имен
- **@parameter_name** - имя формального параметра процедуры
- **default_value** - значение параметра по умолчанию
- **datatype** - тип данных параметра, возможно с указанием размерности
- **SQL_statement** - SQL-оператор, представляющий собой тело процедуры. Может являться группой операторов, объединенных при помощи операторных скобок **begin ... end**.

Пример

```
create procedure nameList( @nameMask char(255) )
as
begin
    select name from sysusers where name like @nameMask
end
```

Создать процедуру можно только в текущей базе. Право на создание процедур по умолчанию принадлежит владельцу базы, который может делегировать это право другим пользователям.

3. Правила именования процедур.

Имя сохраненной процедуры должно быть не более **64** символов длиной. Оно может включать символы **латиницы**, **цифры**, символы **'_'** и **'\$'**. Оно не должно совпадать с именем какого-либо другого объекта в пределах базы.

Имя может быть присвоено как отдельной процедуре, так и их группе. В последнем случае каждой процедуре группы дополнительно присваивается цифровой индекс, указываемый через точку с запятой после имени:

```
create proc users;1
as
select id from sysusers
go
```

```
create proc users;2
as
select name from sysusers
go
```

Подобный групповой способ именования обычно используется автоматическими генераторами процедур. В обычной ситуации лучше давать процедурам индивидуальные осмысленные имена.

4. Уничтожение процедуры.

Сохраненная процедура может быть уничтожена при помощи операции

```
drop proc[edure] [owner.]procedure_name
        [, [owner.]procedure_name [...]]
```

Одним вызовом может быть уничтожено несколько процедур.

Если речь идет о процедурной группе (несколько процедур с одним именем и разными цифровыми индексами), то операция **drop** уничтожает всю группу. Сгруппированные процедуры не могут быть уничтожены по отдельности.

Transact SQL для **ASE** не поддерживает инструкцию **alter procedure** (модификация процедуры), поэтому для изменения сохраненной процедуры ее нужно уничтожить (**drop procedure**) и создать заново (**create procedure**).

Вместе с процедурой уничтожаются все допуски к ней, данные о режимах ее работы, зависимости и т.д. Соответственно, если на процедуру выдавались особые права или устанавливались какие-то дополнительные настройки, при пересоздании процедуры их нужно выдать/установить заново. Поэтому рекомендуется все подобные действия включать прямо в скрипт создания процедуры:

```
/* MyProc.sql */

-- Устанавливаем нужную базу в качестве текущей
use myDatabase
go

-- Если изменяемая процедура существует, уничтожаем ее
if( exists( select 1 from sysobjects where type='P' and name='MyProc' ) )
    drop procedure dbo.MyProc
go

-- Создаем процедуру
create procedure dbo.MyProc
as
begin
    /* Тело процедуры */
end
go

-- Выдаем права и т.д.
grant execute dbo.MyProc to oper_role
go
exec sp_procxmode 'MyProc', 'anymode'
go
```

Поскольку вся "обвязка", выполняемая при пересоздании процедуры, на сервере не сохраняется,

исходные тексты процедур должны храниться в виде файлов на компьютере разработчика и в системе контроля версий.

Хранение процедур только на сервере БД - плохая практика.

5. Запуск сохраненной процедуры.

Синтаксис запуска сохраненной процедуры таков:

```
[exec[ute]] [@return_status = ]  
  [[[server.]database.]owner.]procedure_name[;number]  
  [{ [@parameter_name =] value |  
    [@parameter_name =] @variable [output] }  
  [, { [@parameter_name =] value |  
    [@parameter_name =] @variable [output] }[...]]]  
[with recompile]
```

Например:

```
exec myDd.dbo.someProcedure
```

Ключевое слово **exec** (или **execute**) является необязательным, если вызов процедуры – единственное действие, производимое текущим SQL-пакетом (**batch**):

Если вызывается процедура на текущем сервере, из текущей базы данных, принадлежащая текущему пользователю или **dbo**, соответствующие атрибуты также можно не указывать:

```
use myDB  
go  
someProcedure  
go
```

Вызов процедуры с групповым именем идет с указанием индекса:

```
exec users;2
```

6. Опция with recompile.

Использование опции «**with recompile**» при создании или запуске сохраненной процедуры говорит о том, что процедура должна быть принудительно перекомпилирована.

Если «**with recompile**» используется при **создании**, то процедура будет перекомпилироваться при любом ее вызове. Использование этой опции при **запуске** процедуры приводит к единичной перекомпиляции перед данным конкретным запуском.

Использование «**with recompile**» может быть оправдано в том случае, когда существует возможность существенного изменения объектов, с которыми работает процедура (появление новых индексов, переименование таблиц и т.д.).

Следует помнить, что при использовании этой опции запуск процедур значительно замедляется.

Использование "with recompile" при запуске процедуры

```
create table people( id int, name char(255)
insert into people( id, name ) values( 1, 'Иванов' )
go

-- План процедуры связывается с ID таблицы, а не с ее именем
create procedure listPeople
as
    select id, name from people
go

-- Переименовываем таблицу people и создаем новую с таким же именем
sp_rename people, people_old
create table people( id int, name char(255)
go

-- Выборка идет из people_old, поскольку ее ID указан в плане процедуры
exec listPeople
go

-- Выборка идет из people, поскольку процедура перекомпилируется
exec listPeople with recompile
go
```

Если известно, что переименование и пересоздание таблиц, с которыми работает процедура, это нормальное явление, есть смысл использовать "**with recompile**" прямо при создании процедуры.

Использование "with recompile" при создании процедуры

```
-- Создаем процедуру так, что она будет перекомпилироваться
-- при каждом запуске
create procedure listPeople
as
    select id, name from people
with recompile
go

-- Переименовываем таблицу people и создаем новую с таким же именем
sp_rename people, people_old
create table people( id int, name char(255)
go

-- Выборка идет из people, поскольку процедура перекомпилируется
exec listPeople
go
```


7. Описание и передача параметров.

Сохраненным процедурам при вызове могут передаваться параметры.

Как и в большинстве процедурных языков программирования, для сохраненных процедур разделяют формальные и фактические параметры.

Формальные параметры – это символьные имена параметров, действующие внутри процедуры. Фактические – это те значения или ссылки, которые были переданы в процедуру в момент вызова. Формальные параметры описываются в выражении **create procedure**. При этом указывается их имя, тип данных, размерность и, возможно, значение по умолчанию и вид (входной/выходной).

Пример.

```
create procedure dateAddMonth( @n    int = 1,
                              @dt    datetime,
                              @res    datetime out )
as
begin
    select @res = dateadd( month, @n, @dt )
end
```

В данном примере процедура принимает три параметра. Для параметра **@n** установлено значение по умолчанию **1** (в каких случаях оно используется, будет описано позднее), параметр **@res** является "выходным", т.е., изменения значения данного формального параметра внутри процедуры влияют на значение фактического параметра, переданного в момент вызова.

На самом деле, в Transact SQL не поддерживаются "чистые" выходные параметры. Правильнее говорить не о "входных и выходных параметрах", а о "параметрах, поддерживающих и не поддерживающих возврат значения". Т.е., любой параметр является "входным", а параметры, описанные как **out** (или **output**), работают еще и на "выход".

Поэтому наш пример мог бы выглядеть так:

```
create procedure dateAddMonth( @n    int = 1,
                              @dt    datetime output )
as
begin
    select @dt = dateadd( month, @n, @dt )
end
```

В данном случае через параметр **@dt** передается как начальное значение даты/времени (на вход), так и измененное (на выход).

Максимальное количество параметров сохраненной процедуры - **255**.

Передача параметров возможна как с указанием имен:

```
declare @dat datetime
```

```
select @dat = getdate()  
exec dateAddMonth @dt=@dat output, @n=2
```

так и позиционно:

```
declare @dat datetime  
  
select @dat = getdate()  
exec dateAddMonth 2, @dat output
```

В последнем случае порядок указания фактических параметров должен совпадать с порядком декларации формальных параметров.

Если какой-либо параметр не передан, для него принимается значение по умолчанию. Если значение по умолчанию не указано, будет сгенерирована ошибка.

```
declare @dat datetime  
  
select @dat = getdate()  
exec dateAddMonth @dt=@dat output -- @n
```

8. Возврат статусного значения.

Сохраненная процедура в Transact SQL может возвращать целочисленное статусное значение. Возврат производится при помощи инструкции

return значение

После выполнения этой инструкции выполнение процедуры прерывается. Как уже говорилось, статусное значение может быть только целым числом.

Статусное значение возвращается всегда, даже если в процедуре не используется в явном виде инструкция **return**. Если инструкция **return** не выполнялась, возвращается одно из зарезервированных статусных значений:

0	Выполнение процедуры прошло успешно
-1	Объект не найден
-2	Ошибка типов
-3	Процесс был прерван из-за возникновения «мертвой» блокировки
-4	Ошибка допуска
-5	Синтаксическая ошибка
-6	Различные пользовательские ошибки
-7	Ошибка ресурсов (например, закончилось место на устройстве)
-8	Некритическая внутренняя ошибка
-9	Достигнут системный лимит
-10	Критическое внутреннее рассогласование
-11	Критическое внутреннее рассогласование
-12	Таблица или индекс повреждены
-13	База данных повреждена
-14	Аппаратная ошибка

9. Возврат ResultSet из процедуры.

Сохраненная процедура может формировать один или более наборов данных (**ResultSet**). Для этого достаточно в тексте процедуры написать инструкцию **SELECT** без указания переменных, в которые должны быть помещены результаты выборки:

```
create procedure nameListWithDate( @nameMask char(255) )
as
begin
    select @nameMask as Mask
    select name from sysusers where name like @nameMask
end
go

exec nameList 'ИВ%'
go
```

Вне зависимости от уровня стека вызовов, на котором был сформирован **ResultSet**, он возвращается на клиента. Т.е., **процедура не может обработать ResultSet, возвращенный другой процедурой.**

Также **ResultSet**, сформированный сохраненной процедурой, **не может быть использован для создания курсора.**

10. Правила и ограничения по использованию сохраненных процедур.

Основные правила и ограничения, возникающие при использовании сохраненных процедур (часть из них уже упоминалась ранее):

- Выражение **create procedure** нельзя комбинировать с другими выражениями в одном пакете.
- Текст процедуры может содержать любые типы SQL-инструкций, за исключением инструкции **use** и следующих выражений **create**: **create view**, **create default**, **create rule**, **create trigger**, **create procedure**.
- Внутри процедуры можно создавать и уничтожать другие объекты базы данных. Сразу после создания возможно обращение к этим объектам.
- Внутри процедуры нельзя создать объект, уничтожить его, а затем снова создать объект с тем же именем.
- Объекты, создаваемые внутри сохраненной процедуры, создаются на этапе выполнения этой процедуры, не на этапе ее компиляции.
- Если есть процедура, вызывающая другую процедуру, то вызываемая процедура может обращаться к объектам, созданным первой процедурой.
- Внутри процедуры можно обращаться к временным таблицам.
- Если внутри процедуры создается сессионная временная таблица (имя которой начинается с #), то такая таблица автоматически уничтожается по завершению процедуры. Постоянные временные таблицы не уничтожаются до явного удаления или перезагрузки сервера.
- Максимальное количество параметров процедуры – **255**, максимальное количество локальных переменных ограничено только объемом доступной памяти.

11. Использование сессионных временных таблиц в процедурах.

Некоторые особенности использования сессионных временных таблиц в процедурах возникают в случае, когда процедура должна использовать временную таблицу, созданную за ее пределами.

Так, если есть процедура **ProcExternal**, создающая временную таблицу **#tempTable** и вызывающая после этого процедуру **ProcInternal**, которая должна работать с этой временной таблицей, то при создании **ProcInternal** могут возникнуть сложности.

```
create procedure ProcExternal
as
begin
    create table #tempTable( field1 int )
    exec ProcInternal
    select field1 from #tempTable
    drop table #tempTable -- Не обязательно
end
go

-- Ошибка: таблица #tempTable не найдена
create procedure ProcInternal
as
    insert into #tempTable( field1 ) values( 1 )
go
```

Дело в том, что таблица **#tempTable** будет создана после **запуска** процедуры **ProcExternal**, а не при ее **создании**. Соответственно, на момент создания **ProcInternal** такой временной таблицы не существует.

Проблема решается добавлением в скрипт создания **ProcInternal** блоков создания и удаления временной таблицы, требуемой для работы данной процедуры.

```
-- Создаем временную таблицу, чтоб успокоить транслятор
create table #tempTable( field1 int )
go

-- Все хорошо, процедура создается
create procedure ProcInternal
as
    insert into #tempTable( field1 ) values( 1 )
go
```

12. Понятие функции.

Хранимые функции являются разновидностью хранимых процедур. Они включены в состав программных объектов баз данных с целью наибольшего соответствия языкам программирования. Как и в этих языках программирования, каждая хранимая функция рассматривается в качестве выражения, формирующего одно единственное значение. Хранимые функции применяются для расширения функциональных возможностей операторов **SELECT** и ряда других SQL-операторов.

Отличие от процедуры функция всегда возвращает результат своего действия в место вызова. Процедура ничего не возвращает, а если возвращает, то только через свои аргументы out или in/out.

Оператор **CREATE FUNCTION** несколько отличается от оператора **CREATE PROCEDURE**. Эти отличия состоят в следующем:

- В функциях допустимы только входные формальные параметры (типа **IN**). В связи с этим при описании формальных параметров не используются служебные слова **IN**, **OUT** и **INOUT**.
- После описания формальных параметров функции должен следовать оператор **RETURNS**. Данный оператор указывает тип возвращаемого функцией значения.
- В теле функции обязательно должен присутствовать оператор **RETURN**, который специфицирует возвращаемое функцией значение.
- Функция не может формировать результирующее множество.

Синтаксис создания функции

```
create function [owner_name.] function_name
    ( [ { @parameter_name [as] parameter_datatype [ = default ] }
      [ ,...n ] ] )
    returns return_datatype
[with recompile]
as
[begin]
    function_body
    return scalar_expression
[end]
```

Пример создания функции:

```
CREATE FUNCTION dbo.test( @stroka nvarchar(30) )
returns nvarchar(30)
/* функция приведения строки Серия и Номер к виду 'CVN222' */
as
begin
    declare @otvet nvarchar(30)
    declare @bonus nvarchar(30)
    set @bonus = ''
    if( charindex('Серия',@stroka)!=0 and charindex('№',@stroka)!=0 )
    begin
        WHILE( charindex(' ', @stroka) > 0 )
        BEGIN
            SELECT @stroka = stuff( @stroka, charindex(' ', @stroka), 1, NULL )
        END
        select @bonus = substring(
```

```
        stuff(@stroka,charindex('Серия',@stroka),5,null),  
        1, 3 ) +  
        substring(@stroka,charindex('№',@stroka)+2,6)  
end  
else select @bonus=@stroka  
  
return @bonus  
end
```

Вызов функции в команде SELECT - `select dbo.test('СерияCV №2222')`

Ранее созданные функции хранятся в базах данных до тех, пока они не удаляются посредством оператора **DROP FUNCTION**.

13. Понятие триггера.

Триггер – это сохраненная процедура, ассоциированная с таблицей, автоматически срабатывающая при внесении в эту таблицу каких-либо изменений (**INSERT**, **UPDATE**, **DELETE**).

Запуск триггеров производится автоматически. Они вызываются вне зависимости от того, что вызвало модификацию данных.

Триггер ассоциируется с одним или более типов модификации данных (**INSERT**, **UPDATE**, **DELETE**) и запускается один раз для каждого SQL-выражения, вызывающего этот тип модификации для ассоциированной таблицы.

Запуск триггера производится только после того, как модификация данных завершится и сервер проверит все правила и ограничения, накладываемые на модифицируемую таблицу.

Триггер выполняется в той же транзакции, что и модифицирующее выражение. Таким образом, все выполненные модификации могут быть отменены из триггера запуском инструкции **rollback transaction**.

Наиболее распространенные задачи, решаемые при помощи триггеров:

- **Каскадирование изменений (из практики - лучше так не делать).** Например, триггер, срабатывающий при удалении записей из таблицы, может выполнять удаление из связанных таблиц.
- **Поддержание целостности данных.** Например, если есть справочник авторов и таблица написанных ими книг, триггер, срабатывающий при вставке данных в таблицу книг может проверять, существует ли соответствующий автор и, в случае его отсутствия, откатывать транзакцию.
- **Проверка значений столбцов.** Триггер может осуществлять сложные проверки, связанные с обращением к другим таблицам.
- Триггер может производить **сравнение состояния данных до и после модификации** и, основываясь на этом, выполнять какие-либо действия.

14. Создание и уничтожение триггера.

Создание триггера производится при помощи инструкции **create trigger**:

```
create trigger [owner.]trigger_name  
  on [owner.]table_name  
  for {insert, update, delete}  
  as SQL_statements
```

В разделе **on** указывается имя таблицы, с которой ассоциируется триггер, в разделе **for** – список типов модификаций данных, на которые этот триггер будет реагировать (**insert**, **update**, **delete**).

Уничтожение триггера (или нескольких триггеров) производится при помощи инструкции **drop trigger**:

```
drop trigger [owner.]trigger_name [, [owner.]trigger_name]
```

15. Таблицы **inserted** и **deleted**, кол-во изменяемых строк.

Для того, чтобы триггер мог получить информацию об изменяемых данных, служат две виртуальные таблицы: **inserted** и **deleted**.

Inserted содержит новые данные (добавляемые для команды **insert** или новое состояние для **update**), **deleted** – предыдущее состояние (удаляемые записи для команды **delete** или состояние до начала операции для **update**). Структура этих таблиц совпадает со структурой основной таблицы, ассоциированной с триггером.

Таблицы **inserted** и **deleted** могут использоваться в запросах, выполняемых триггером, на правах обычных таблиц. Например триггер, выполняющий проверку того, что для добавляемых (или изменяемых) счетов корректно указан код валюты, может выглядеть так:

```
create trigger Account_InsertUpdate on Accounts
for insert, update
as
begin
    if( exists( select 1 from inserted i
                where not exists( select 1 from Currency c
                                where c.Currency = i.Currency ) )
    begin
        rollback transaction
        raiserror 25001 'Wrong currency'
    end
end
```

Как уже говорилось, триггер вызывается один раз для каждой операции изменения таблицы. Т.е., даже если в результате выполнения SQL-инструкции добавляется (изменяется, удаляется) несколько строк, триггер будет вызван один раз. При этом таблицы **inserted** и/или **deleted** будут содержать столько строк, сколько подверглось изменению (добавлению, удалению).

Узнать, сколько строк было изменено (добавлено, удалено), можно, проанализировав глобальную переменную **@@rowcount**. В приведенном ниже примере для накопления общего количества продаж используются либо данные поля единичной записи, либо сумма с группировкой:

```
create trigger salesdetail_insert on salesdetail for insert
as
begin
    if( @@rowcount = 1 )
    begin
        update titles
            set total_sales = t.total_sales + i.qty
            from inserted i, titles t
            where t.title_id = i.title_id
    end
else
```

```
begin
  update titles
    set total_sales = t.total_sales +
      ( select sum( i.qty ) from inserted i
        where i.title_id = t.title_id )
    from titles t
  end
end
```

16. Обработка изменения конкретных полей таблицы.

При создании триггера возможен расширенный вариант синтаксиса, позволяющий обрабатывать изменения конкретных полей таблицы:

```
create trigger [owner.]trigger_name on [owner.]table_name
for {insert, update}
as
    if update (column_name) [{and | or} update (column_name)][...]
        SQL_statements
    [if update (column_name) [{and | or} update (column_name)][...]
        SQL_statements][...]
```

При использовании выражений **if update(имя_столбца)** в триггере, обрабатывающем **insert**, значение «истина» будет возвращаться только в том случае, когда поле явно перечислено в списке полей или для него задано значение по умолчанию. При неявной вставке значения **NULL** соответствующая ветка **if update** не сработает.

Например:

```
create table t( a int null, b int null )
go

create trigger t_insert on t for insert
as
    if update( a ) print 'Fired'
go

insert into t( a, b ) values( 1, 2 )      -- триггер сработает
insert into t( a, b ) values( NULL, 3 )  -- триггер сработает
insert into t( b )      values( 4 )      -- триггер НЕ сработает
```

17. Отмена действия триггера.

Действие триггера может быть отменено при помощи инструкции **rollback trigger** или **rollback transaction** (если триггер выполняется в рамках транзакции).

При этом следует учитывать, что **rollback trigger** отменяет только действие триггера и того выражения, которое привело к его запуску. **Rollback transaction** «откатывает» всю транзакцию.

```
begin transaction
insert into people( id ) values( 9999 )
insert into people( id ) values( 9998 )
commit transaction
```

В приведенном примере, если при выполнении второй инструкции **insert** триггер, связанный с таблицей **people**, запустит **rollback trigger**, отменится только эта инструкция, первая вставка отработает. Если же этот триггер выполнит **rollback transaction**, отменится все.

Синтаксис выражения **rollback trigger**:

```
rollback trigger [with raiserror_statement]
```

Например:

```
rollback trigger with raiserror 25001 'Something wrong :('
```

18. Запрет триггеров.

Как **bcp**, так и инструкция **insert**, вызывают запуск триггеров при вставке данных в ассоциированные с ними таблицы. Естественно, это приводит к замедлению выполнения операций (bcp даже переключается в «медленный» режим копирования).

Для того, чтобы отключить срабатывание триггеров при загрузке в таблицу больших объемов данных, можно использовать опцию **disable trigger** команды **alter table**:

```
alter table [database_name.[owner_name].]table_name
    {enable | disable} trigger [trigger_name]
```

Вновь включить запуск триггеров можно при помощи опции **enable trigger**.

Если имя триггера не указано, действие команды распространяется на все триггеры, ассоциированные с данной таблицей.

Этим средством может воспользоваться только владелец таблицы или администратор базы.

19. Вложенные и рекурсивные запуски триггеров.

Триггеры допускают вложенные запуски. Вложенность возникает, если триггер модифицирует таблицу, с которой в свою очередь также связан триггер.

Текущий уровень вложенности можно узнать, проанализировав значение переменной **@@nestlevel**. Максимальный уровень вложенности для триггеров – **16**.

Возможность вложенных запусков триггеров можно включать и выключать, изменяя значение параметра **allow nested triggers**.

Если один из «вложенных» триггеров выполняет **rollback trigger**, отменяется действие всех вышестоящих триггеров, включая первоначальную инструкцию, вызвавшую срабатывание первого триггера.

По умолчанию триггер не может рекурсивно вызвать сам себя. Таким образом, если, например, триггер, реагирующий на update, производит модификацию ассоциированной с ним таблицы, повторный запуск этого триггера не производится.

Однако это правило можно изменить, включив опцию **self_recursion** команды **set**. При этом параметр **allow nested triggers** также должен быть включен.

Действие опции **self_recursion** распространяется только на текущую клиентскую сессию. Если опция включается внутри триггера, ее действие ограничено данным триггером, причем если этот триггер вызывает срабатывание другого триггера, значение опции автоматически сбрасывается в **off**.

При включенной опции **self_recursion** триггер может запускать сам себя рекурсивно. Уровень вложенности при этом не должен превышать **16**.

20. Команда **set** в триггере, переименование объектов.

Если в триггере используется команда **set**, ее действие распространяется только на время выполнения данного триггера. По окончании параметр, измененный командой **set**, устанавливается в свое первоначальное значение.

Для ASE версий до 15.0 при переименовании объекта, к которому обращается триггер, этот триггер должен быть пересоздан. Здесь ситуация аналогична тому, как ведут себя сохраненные процедуры при переименовании объектов, к которым они обращаются.

Для ASE 15.0 и старше связи между объектами при переименовании изменяются автоматически.

Наиболее безопасная политика – не переименовывать объекты, к которым обращаются триггеры.

21. Ограничения использования триггеров.

ASE вводит следующие ограничения для триггеров:

- С таблицей может быть ассоциировано не более трех триггеров (по одному на **insert**, **update**, **delete**)
- Триггер может быть ассоциирован только с одной таблицей. При этом один триггер может обрабатывать все три типа модификаций (**insert**, **update**, **delete**)
- Нельзя создать триггер для временной таблицы или представления (**view**). При этом сам триггер может обращаться к представлениям или временным таблицам
- Инструкции **writetext** и **truncate table** не активируют триггеров
- Нельзя создать триггер для системной таблицы
- Триггер не может получать данные типа **text** или **image** из виртуальных таблиц **inserted** и **deleted**
- При использовании **Component Integration Services (CIS)** триггеры, созданные для **проxy-таблиц**, обладают ограниченной функциональностью. Они могут быть созданы и запущены, однако не могут использовать таблиц **inserted** и **deleted**, поскольку эти таблицы используют лог транзакций, а для проxy-таблиц изменения фиксируются в логе транзакций удаленного сервера.

Поскольку триггер выполняется как часть транзакции, в нем не допускаются следующие выражения:

- Все команды **create ...** (create database, create table, create index, create procedure, create default, create rule, create trigger, и create view)
- Команды **drop ...**
- **Alter table** и **alter database**
- **truncate table**
- **grant** и **revoke**
- **update statistics**
- **reconfigure**
- **load database** и **load transaction**
- **disk ...** (disk init, disk mirror, disk refit, disk reinit, disk remirror, disk unmirror)
- **select into**

Право создания триггера для какой-либо таблицы есть только у владельца этой таблицы. Это право не может быть никому делегировано.

Если триггер производит какие-либо действия с объектами, к которым создатель триггера не имеет допуска, ASE позволит создать такой триггер, однако модификации ассоциированной с триггером таблицы окажутся заблокированными, поскольку во время выполнения триггера будет возникать ошибка допуска, что приведет к отмене транзакций.

22. Получение информации о триггерах.

Так же, как и остальные объекты базы, триггеры перечислены в таблице **sysobjects**. Поле **type** этой таблицы для триггеров содержит значение **'TR'**. Таким образом, для получения списка триггеров в базе можно воспользоваться выражением

```
select * from sysobjects where type = 'TR'
```

Также можно воспользоваться системными процедурами:

- **sp_help** – для получения общей информации о триггере и выяснения его статуса (разрешен/запрещен)
- **sp_helptext** – для просмотра исходного текста триггера (при наличии соответствующего допуска)
- **sp_depends** – для получения списка триггеров, обращающихся к данному объекту или списка объектов, к которым обращается данный триггер

23. Триггеры и производительность.

«Накладные расходы» на запуск триггера обычно невелики. Основную нагрузку, создаваемую триггером, определяют действия, производимые в нем, таблицы, к которым триггер обращается.

При использовании триггеров следует максимально «облегчать» действия производимые в них. При необходимости выполнять какие-либо сложные операции стоит подумать о построении асинхронной схемы обработки (например, триггер только добавляет записи в какой-то промежуточный журнал, который впоследствии обрабатывается внешним процессом).

Всегда следует помнить, что триггеры выполняются в рамках транзакции и обработка в них больших объемов данных могут привести к переполнению лога транзакций, возникновению трудно локализуемых «мертвых» блокировок и т.д.